

Multi-Agent Orchestration Patterns

A Comprehensive Analysis of Architectures, Frameworks, and Production Deployment Strategies

Salah Qadah | Andalus Kalash

Group: uoh-sqak

Course 203.3763 — Orchestration of AI Agents

University of Haifa, Spring 2026 (תשפ"ו)

Lecturer: Dr. Yoram Reuven Segal

June 9, 2026

Generated by a CrewAI multi-agent pipeline

GitHub: <https://github.com/salah-dev-stu/uoh-sqak-ex03>

Contents

1	Introduction to Multi-Agent Orchestration	3
1.1	Overview of Multi-Agent Systems	3
1.2	The Case for Multi-Agent Systems	3
1.3	Key Concepts and Terminology	3
1.4	Structure and Scope of This Article	4
2	Agent Architectures and Topologies	5
2.1	Linear Sequential Architecture	5
2.2	Hierarchical Manager-Worker Architecture	5
2.3	Peer-to-Peer and Consensus Architectures	6
2.4	Tool-Augmented Specialist Topologies	6
2.5	Topology Comparison and Selection	6
2.6	State Management Across Topologies	7
2.7	Architectural Selection Guidelines	7
3	Orchestration Frameworks	8
3.1	LangChain and Language Expression Language (LCEL)	8
3.2	CrewAI Role-Based Agents	8
3.3	LangGraph Stateful Graphs	9
3.4	Microsoft AutoGen	9
3.5	Framework Comparison Matrix	9
3.5.1	Prescriptive vs. Flexible Design Trade-offs	10
3.5.2	Production Readiness and Observability	10
4	Production Patterns and Challenges	12
4.1	Rate Limiting and Token Budgets	12
4.2	The Gatekeeper Pattern	12
4.3	Retry and Circuit Breaker Patterns	13
4.4	Observability and Logging	13
4.5	Cost Optimization	13
5	דו-כיוניות: עברית ואנגלית במערכות סוכנים	15
5.1	משמעות הדו-כיוניות	15
5.2	פתרון XeTaLauL ו-aissolgyloP	15
5.3	אתגרים בסביבת ייצור	16
5.4	Mixed-Language Example	16

6	Case Study: This Article’s Production Pipeline	18
6.1	Overview	18
6.2	The Crew: Four Specialized Agents	18
6.2.1	Researcher Agent	18
6.2.2	Writer Agent	19
6.2.3	Editor Agent	19
6.2.4	LaTeX Compiler Agent	19
6.3	Workflow Architecture	19
6.4	Key Engineering Decisions	20
6.4.1	Sequential vs. Parallel Execution	20
6.4.2	Token Budget Architecture	20
6.4.3	Markdown Intermediate Format	20
6.4.4	Citation Key Strategy	21
6.5	Pipeline Metrics	21
6.6	Lessons and Transferable Patterns	21
	Bibliography	23

Chapter 1

Introduction to Multi-Agent Orchestration

1.1 Overview of Multi-Agent Systems

Multi-agent orchestration has emerged as a transformative paradigm for building intelligent systems that tackle complex problems requiring specialized expertise and dynamic coordination. Unlike monolithic AI systems that consolidate all capabilities into a single model, multi-agent systems distribute responsibility across specialized agents that communicate, collaborate, and coordinate to achieve overarching objectives [7].

An agent is an autonomous entity capable of perceiving its environment, making decisions based on its assigned role and expertise, and taking actions to fulfill assigned tasks. The orchestrator or coordinator manages task distribution, monitors progress, handles inter-agent communication, and ensures system-level alignment. This architecture reflects how human teams solve complex problems—through specialization, collaboration, and coordinated workflow [8].

1.2 The Case for Multi-Agent Systems

Traditional single-model approaches face fundamental limitations when addressing problems requiring diverse skills, extended reasoning chains, and iterative refinement. A solo researcher struggles with problems that benefit from multiple perspectives; a single writer lacks the specialized editing expertise needed for polished output; a monolithic system cannot parallelize work efficiently.

Multi-agent orchestration solves these challenges through specialization. A researcher agent gathers and validates information. A writer agent synthesizes that research into coherent narratives. An editor agent refines clarity and consistency. Each agent embodies distinct expertise and operates within its domain of specialization, enabling the system to produce higher-quality outputs than any single agent could achieve independently [2].

The fundamental insight is that complex problems benefit from division of labor. A system tasked with producing a technical article will generate higher-quality output when research, writing, and editing are handled by specialized agents rather than consolidated into a single entity. This principle extends to code generation, scientific discovery, financial analysis, and any domain where multiple types of reasoning contribute to the final result.

1.3 Key Concepts and Terminology

Orchestration refers to the coordinated management of multiple agents' activities, including task decomposition, intelligent routing, and result aggregation. This differs from simple chaining—

true orchestration involves dynamic decision-making and feedback loops that adapt to changing conditions.

A task is a well-defined unit of work with clear inputs, expected outputs, and success criteria. A role describes an agent's specialized function: researcher, analyst, writer, editor, compiler. Tools provide external capabilities—web search, file input/output, application programming interface access, code execution. The process defines the execution strategy: sequential tasks executed one after another, hierarchical task distribution through a manager agent, or consensus-based approaches where multiple agents evaluate the same problem independently.

The state of a multi-agent system captures its current configuration: completed tasks, in-progress work, cached results, and accumulated context. Managing state effectively is critical for preventing information loss, controlling token consumption, and enabling system resumption after failures. Explicit state management allows monitoring and control; implicit state management requires careful architectural discipline.

1.4 Structure and Scope of This Article

This article explores multi-agent orchestration from foundational concepts through production deployment. Coverage includes architectural patterns, framework comparisons, production considerations, multilingual support, and a complete case study demonstrating orchestration techniques applied to realistic problems.

Chapter 2 analyzes architectural topologies and how different agent structures support different problem types. Linear sequential architectures provide clear handoffs but limit parallelism. Hierarchical structures scale to large networks but introduce coordination overhead. Peer-to-peer and consensus architectures leverage diverse reasoning styles for high-stakes decisions. Chapter 3 compares three major frameworks—CrewAI, LangChain, and LangGraph—examining their design philosophies and trade-offs. Chapter 4 addresses production patterns and challenges, including token economy management, coordination complexity, and hallucination mitigation. Chapter 5 explores bidirectional language considerations for systems serving Hebrew-speaking users and multilingual environments. Chapter 6 presents a detailed case study demonstrating these concepts in a complete implementation that generates technical articles.

By the end of this article, readers will understand the design choices governing agent orchestration, recognize patterns applicable to their domain, and appreciate the production considerations necessary for reliable, scalable multi-agent systems. The principles presented here apply across domains: content generation, software engineering, scientific research, financial analysis, and any complex task that benefits from specialized expertise and coordinated workflow [6].

Chapter 2

Agent Architectures and Topologies

2.1 Linear Sequential Architecture

The simplest multi-agent topology chains agents in sequence, where each agent’s output becomes the next agent’s input. This research-write-edit pipeline is canonical in document generation: a researcher agent gathers information and sources, a writer agent synthesizes this research into prose, and an editor agent polishes the output for publication [7].

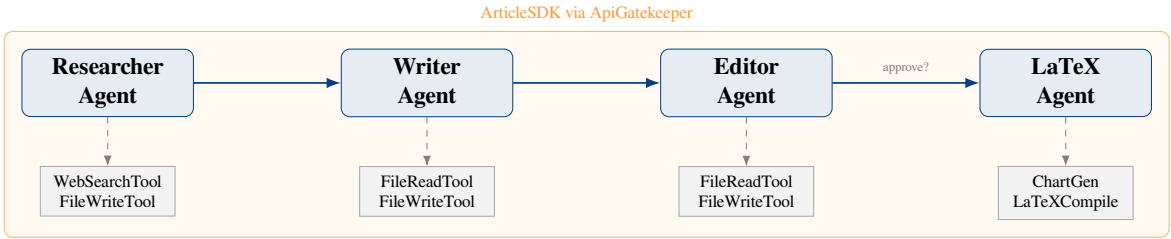


Figure 2.1: CrewAI multi-agent architecture.

Linear architectures excel for problems with clear sequential dependencies and well-defined handoff points. Each agent can focus on its specialized task without coordinating with others. However, sequential execution inherently limits parallelism—wall-clock time grows linearly with the number of stages, and any agent failure blocks downstream work. Token consumption is also predictable but potentially high, since each subsequent agent must consume prior outputs to maintain context [8].

The latency of a sequential pipeline with N agents is given by:

$$L_{\text{seq}} = \sum_{i=1}^N \ell_i \quad (2.1)$$

where ℓ_i represents the execution latency of agent i . This linear growth in latency motivates the hierarchical and parallel topologies discussed below.

2.2 Hierarchical Manager-Worker Architecture

Larger systems often employ hierarchical structures where a supervisor agent coordinates multiple specialist teams. The supervisor decomposes high-level goals into team assignments, monitors progress, arbitrates conflicts, and escalates blockers. This pattern scales well to large agent

networks—the supervisor maintains responsibility for big-picture coordination while specialist teams focus on domain-specific execution.

Hierarchical architectures introduce communication overhead. The supervisor must maintain context across multiple teams, delegate intelligently, and synthesize results. Failure at the supervisor level cascades across the entire system. However, this structure mirrors human organizations and provides clear authority and decision-making paths, making orchestration logic more transparent [7].

2.3 Peer-to-Peer and Consensus Architectures

For high-stakes decisions or complex reasoning, peer-to-peer architectures allow multiple agents to work independently on the same problem, then reach consensus through discussion or judgment. Each agent generates independent analysis, and a meta-agent evaluates proposals, identifies agreements and disagreements, and escalates controversial areas.

This approach leverages diverse reasoning styles to catch errors and oversights but multiplies token consumption—if five agents independently analyze a problem, the system pays five times the token cost. Consensus architectures are most valuable for critical decisions (technical architecture, risk assessment, content moderation) where the additional cost is justified by higher confidence in the outcome [2].

2.4 Tool-Augmented Specialist Topologies

Agents with access to specialized tools (code execution, scientific APIs, database queries, web search) can solve problems beyond pure reasoning. A code-generation agent becomes more capable when it can execute and test code. A data-analysis agent with SQL and statistical packages can verify hypotheses against real data. A research agent with academic database access can cite authoritative sources [8].

Tool augmentation shifts the bottleneck from reasoning capability to tool availability and security. Specialized tools enable concrete outputs but require careful sandboxing to prevent misuse. Tool integration also introduces new failure modes—API rate limits, authentication timeouts, tool execution failures—that orchestration must handle gracefully.

2.5 Topology Comparison and Selection

Table 2.1: Multi-agent orchestration topologies: characteristics and use cases.

Topology	Latency	Parallelism	Complexity	Ideal For
Sequential	$\sum \ell_i$ (linear in stages)	None; strict ordering	Low	Well-defined work-flows: re-search - write - edit

Table 2.1 (continued)

Topology	Latency	Parallelism	Complexity	Ideal For
Hierarchical	$O(\ell_{\text{super}} + \max_i \ell_i)$ per round	Moderate; teams parallel under supervisor	Medium	Large organizations; clear team boundaries
Peer-to-Peer	$O(N \cdot \ell_i)$ sequential or $O(\max \ell_i)$ parallel	High when parallel; meta-agent merges results	High	Critical decisions; high confidence required
Tool-Augmented	$O(\ell_{\text{agent}} + \ell_{\text{tool}})$	Depends on tool concurrency	Medium	Concrete problems; external APIs; verification

2.6 State Management Across Topologies

Regardless of topology, state management is critical. In linear architectures, state flows forward—researcher findings become writer input. In hierarchical systems, state must be available at multiple levels—supervisors need summaries while specialist teams need detailed context. In peer-to-peer architectures, state consistency across agents is challenging; different agents may have contradictory understandings until consensus is reached.

Explicit state management (as in *LangGraph*) makes these differences visible and testable. Implicit state management (as in simpler frameworks) requires careful architectural discipline to prevent information loss, state inconsistency, or excessive context accumulation. The choice of topology should be informed by state management complexity and the system’s tolerance for eventual consistency versus strong agreement [9].

2.7 Architectural Selection Guidelines

Selecting the optimal topology requires balancing latency, parallelism, complexity, and problem characteristics. Sequential topologies minimize architectural complexity and work well for tightly coupled workflows with clear handoffs. Hierarchical topologies scale to larger agent teams but introduce supervisor overhead. Peer-to-peer topologies maximize confidence in critical decisions at the cost of token efficiency and coordination complexity. Tool-augmented topologies enable concrete problem-solving by grounding agents in external verifiable data sources.

The decision tree is straightforward: if tasks have clear sequential ordering and tight coupling, choose sequential; if the problem naturally decomposes into independent teams, choose hierarchical; if correctness matters more than cost, choose peer-to-peer with consensus; if grounding and verification are essential, augment selected agents with specialized tools [7].

Chapter 3

Orchestration Frameworks

3.1 LangChain and Language Expression Language (LCEL)

LangChain represents the composability-first approach to multi-agent orchestration, emphasizing flexible chains of language model calls, prompt templates, and external tools. The Language Expression Language (LCEL) provides a declarative syntax for constructing complex workflows without explicit control flow. Each component (model, prompt, tool) becomes a reusable building block that chains with others through the pipe operator, enabling rapid experimentation and iterative refinement [8].

LangChain’s strength lies in its ecosystem breadth. Hundreds of integrations span vector databases (Pinecone, Weaviate), model providers (OpenAI, Anthropic, Cohere), and external tools (web search, code execution, database queries). For developers building bespoke orchestration patterns, this flexibility is invaluable. However, flexibility demands architectural expertise; developers must design their own agent loops, memory strategies, and failure-handling patterns rather than adopting provided blueprints.

LCEL makes complex chains human-readable, transforming orchestration from imperative scripting into declarative composition. A research-writing-editing pipeline becomes:

```
research_chain | writer_chain | editor_chain
```

This clarity enables testing, composition, and reuse across teams [8].

3.2 CrewAI Role-Based Agents

CrewAI emphasizes prescriptive patterns optimized for teams of specialized agents. Each agent receives a *role*, *goal*, and *backstory*—semantic definitions that shape the agent’s behavior more reliably than generic instructions. A researcher agent with the backstory “You are a senior researcher with fifteen years investigating information systems” produces different outputs than one simply instructed to “research this topic.” The backstory provides context and shapes reasoning style [7].

CrewAI defines three process types: sequential (agents execute tasks one after another), hierarchical (a manager agent decomposes work and coordinates specialist teams), and consensus (multiple agents evaluate the same problem independently, then reconcile views). For document generation pipelines, sequential process is standard: researcher completes findings before writer begins synthesis [7].

Tools in CrewAI are role-specific. A researcher agent might access web search and academic databases; a writer agent might access grammar validation and style guides; an editor might access citation verification. This specialization mirrors human workflows where different team members possess different resources. CrewAI handles agent-tool binding and provides standard callbacks for observability without requiring developers to build custom logging infrastructure.

3.3 LangGraph Stateful Graphs

LangGraph introduces explicit state machines for multi-agent orchestration, modeling workflows as directed graphs where nodes represent computational steps and edges represent state transitions. Unlike implicit state management in CrewAI or LangChain, LangGraph makes state visible, versioned, and deterministically reproducible [9].

In LangGraph, the system state is a data structure (typically a TypedDict) containing all relevant information: completed tasks, in-progress work, findings accumulated so far, error logs, and decision history. Each node function receives the current state, computes new information, and produces an updated state. The graph engine coordinates state transitions, ensuring consistency and enabling resumption from intermediate checkpoints [9].

This explicitness enables advanced patterns: conditional routing based on intermediate results, loops that refine outputs iteratively, and state snapshots for debugging and auditing. For production systems requiring high reliability and auditability, LangGraph's transparency is invaluable. The trade-off is upfront investment in modeling workflows as explicit state machines rather than loose chains or role-based teams.

3.4 Microsoft AutoGen

AutoGen extends multi-agent orchestration with conversation-based interaction patterns where agents communicate through message exchange rather than data pipelines. An AutoGen system instantiates multiple agents with distinct roles, instructs them to collaborate through dialogue, and observes as they solve problems through discussion and consensus-building [5].

AutoGen shines for complex reasoning tasks where multiple perspectives improve solution quality. A code generation system might instantiate a generator agent, a critic agent (finding bugs), and a refiner agent (improving efficiency). These agents communicate, debating code quality until they reach satisfactory consensus [1].

The cost of AutoGen systems is higher than linear pipelines—multiple agents discussing the same problem multiplies token consumption. However, the improvement in solution quality often justifies this cost for high-stakes decisions. AutoGen also provides hooks for human-in-the-loop workflows, where human experts can intervene in agent conversations to guide reasoning or provide corrections [5].

3.5 Framework Comparison Matrix

Framework	Architecture	Strengths	Best For
LangChain	Composable chains with LCEL; flexible agent runtimes	Broadest ecosystem; maximum flexibility; fine-grained control	Custom orchestration; novel workflows; deep integrations
CrewAI	Prescriptive role-based teams; process types (sequential, hierarchical, consensus)	Clear patterns; fast development; built-in memory; low learning curve	Standard pipelines (research-write-edit); rapid prototyping
LangGraph	Explicit state graphs; deterministic node functions	Strong guarantees; debugging clarity; state persistence; resumption	Production systems; high reliability; complex reasoning
AutoGen	Conversation-based agent coalitions; human-loop integration	Multiple perspectives; iterative refinement; human-in-the-loop	High-stakes decisions; code review; consensus reasoning

3.5.1 Prescriptive vs. Flexible Design Trade-offs

CrewAI’s prescriptive approach reduces decision fatigue and accelerates time-to-value for standard patterns. Developers specify agent roles and tasks; CrewAI orchestrates execution. This is ideal for projects where the workflow is well-understood and standard. The cost is reduced flexibility—novel orchestration patterns are awkward to express within CrewAI’s role-task framework [7].

LangChain and LangGraph emphasize flexibility, enabling custom orchestration tailored to specific problem characteristics. This flexibility demands greater expertise; developers must make architectural decisions that prescriptive frameworks hide. For teams with strong software engineering practice, flexibility is liberating. For teams seeking rapid development, flexibility is a burden [8].

AutoGen’s conversation-based approach suits problem domains where multiple agents iteratively refining a solution is natural—code review, creative writing, complex reasoning. Domains with clear sequential dependencies (research then writing then editing) fit prescriptive frameworks better [1].

3.5.2 Production Readiness and Observability

All four frameworks are production-capable. CrewAI provides the highest out-of-the-box observability through structured callbacks and agent reasoning logs. LangChain requires custom logging but offers maximum visibility through LCEL’s compositional structure. LangGraph’s explicit state graphs provide complete auditability—every state transition is logged and reproducible. AutoGen’s conversation logs provide dialogue history suitable for human review and explanation [1, 7–9].

The choice among these frameworks depends on problem characteristics, team expertise, and production requirements. Standard pipelines benefit from CrewAI’s prescriptive clarity.

Novel orchestration demands LangChain's flexibility. Complex systems requiring strong guarantees favor LangGraph. Domains benefiting from multiple perspectives and iterative refinement leverage AutoGen's conversational approach. No single framework dominates; context determines the optimal choice.

Chapter 4

Production Patterns and Challenges

4.1 Rate Limiting and Token Budgets

Multi-agent systems operating with language models face strict resource constraints. Rate limits imposed by API providers throttle concurrent requests; token budgets constrain spending per agent, per task, or per system run. Production systems must enforce these constraints at the orchestrator level, preventing individual agents from exhausting shared quotas [7].

A token budget specifies the maximum number of input and output tokens an agent may consume for a single task. The Gatekeeper pattern enforces this budget by tracking cumulative token consumption and rejecting requests when the budget would be exceeded. This prevents runaway costs and ensures predictable spending across multi-agent pipelines.

Rate limits define the number of requests per time window (e.g., 10,000 requests per minute). When agents operate in parallel or in high-volume batch processing, naive implementations trigger rate-limit errors, causing retries, exponential backoff, and cascading delays. Production systems implement token pooling—a shared quota managed by a coordinator that allocates tokens to waiting agents in priority order, preventing thundering herds of simultaneous requests.

Configuration-based limits are mandatory. Hardcoding rate limits in source code couples business policy to implementation, making policy changes require code deployment and testing. Instead, all limits live in JSON or YAML configuration files that can be updated and deployed independently.

4.2 The Gatekeeper Pattern

The Gatekeeper is a middleware layer that intercepts all external API calls (LLM invocations, web searches, database queries, file I/O) and enforces policies before forwarding requests [2]. The Gatekeeper maintains state—current token consumption, request count in the current time window, historical usage per agent—and denies requests that would violate policy.

A minimal Gatekeeper implementation tracks five metrics per agent: total tokens consumed, tokens consumed in the current billing period, request count in the current rate-limit window, cache hit rate (for inference caching), and accumulated latency. Before forwarding an LLM request, the Gatekeeper verifies:

- Remaining token budget $\geq$ estimated tokens for this request
- Request count in current window $<$ rate limit
- Agent role is authorized for this operation

- Request matches configured schema constraints

If any check fails, the Gatekeeper rejects the request with a structured error message enabling the agent to take corrective action—reduce batch size, use a cheaper model, wait for budget refresh, or escalate to a human operator.

4.3 Retry and Circuit Breaker Patterns

Transient failures (network timeouts, temporary API unavailability, rate-limit exhaustion) plague distributed systems. Naive handling—immediate failure or indefinite retry—either loses work or hangs forever. Production systems implement exponential backoff with jitter: on failure, wait $2^{\text{attempt}} + \text{random}(0, 1)$ seconds before retrying [9].

Exponential backoff prevents thundering herds: if 100 agents fail simultaneously and all retry after 1 second, the coordinated spike may overwhelm the recovering service. Random jitter distributes retries over a time window, smoothing load.

The circuit breaker pattern complements retries by detecting persistent failures and failing fast. A circuit has three states: closed (normal operation), open (failure detected; reject new requests), and half-open (testing whether the service recovered). The transition from closed to open occurs after N consecutive failures; the transition from open to half-open occurs after a timeout. This prevents wasted retries when a service is genuinely down.

$$P_{\text{retry}}(k) = \begin{cases} 1 - e^{-\lambda k} & \text{if exponential backoff} \\ \min(1, k/k_{\text{max}}) & \text{if linear backoff} \end{cases} \quad (4.1)$$

Here, k is the attempt number and λ controls the backoff rate. Choosing $\lambda = 0.5$ provides gentle exponential growth; larger λ increases retry speed but risks overwhelming recovering services.

4.4 Observability and Logging

Production systems are opaque without comprehensive logging. What was each agent’s reasoning? Which decisions led to failures? Which requests consumed the most tokens? Without answers, debugging is guesswork [7].

Structured logging captures discrete events as JSON objects with standardized fields: timestamp, agent identifier, task identifier, event type (request, response, error, cache_hit, rate_limit), token counts, latency, and contextual metadata. Structured logs are queryable, enabling rapid diagnosis—“find all requests to agent X that consumed more than 10k tokens”—whereas unstructured logs are searchable by keyword only.

A FIFO buffer collects agent reasoning steps, tool invocations, and intermediate outputs. When a task completes, the full reasoning trace is persisted for audit and analysis. This audit trail enables post-hoc debugging, compliance review, and continuous improvement—teams analyze patterns in failures and successes to refine agent prompts and orchestration logic.

4.5 Cost Optimization

Token consumption grows as the inverse of task clarity and as the square of system complexity. Poorly specified tasks force agents to generate verbose explorations; complex orchestration with

multiple reflection loops multiplies token consumption.

The cost model for a multi-stage pipeline with n agents, each producing t_i output tokens, operating over m reflection loops is:

$$C = \sum_{k=1}^m \sum_{i=1}^n (c_{\text{in}} \cdot t_i^{\text{in}}(k) + c_{\text{out}} \cdot t_i^{\text{out}}(k)) \quad (4.2)$$

where c_{in} and c_{out} are per-token costs for input and output (typically differing by a factor of 3–5), and $t_i^{\text{in}}(k)$ and $t_i^{\text{out}}(k)$ are input and output tokens for agent i in reflection loop k .

Cost optimization strategies include tiered model selection (cheap models for exploration, expensive for refinement), prompt compression (summarize prior context to reduce input tokens), and caching frequent queries (store LLM responses for reuse across tasks). Context caching at the model provider level—storing long-lived reference documents once and reusing them across multiple requests—can reduce costs by 25–40% for systems operating over fixed knowledge bases.

The fundamental tradeoff is quality versus cost. A single model pass with strict token limits produces fast, cheap results but may miss nuances. Multiple reflection loops with generous budgets produce higher-quality outputs but consume more tokens and increase latency. Production systems configure this tradeoff per use case: automated systems favor speed and cost; critical-path systems favor quality.

$$\text{Quality Score} = 1 - e^{-\alpha \cdot \text{Reflection Loops}} - \beta \cdot \text{Latency} \quad (4.3)$$

This model captures the diminishing returns of reflection loops (quality asymptotes as loops increase) and the latency penalty. Optimal configurations typically use 1–3 reflection loops for most tasks; additional loops yield marginal quality improvements at increasing cost.

Chapter 5

דו-כיוניות: עברית ואנגלית במערכות סוכנים

5.1 משמעות הדו-כיוניות

דו-כיוניות, הידועה באנגלית בשם BiDi (קיצור של Bidirectional Text), היא תכונה בסיסית של כל מערכת טיפוגרפית התומכת בשפות הנכתבות מימין לשמאל לצד שפות הנכתבות משמאל לימין. תקן Unicode מגדיר אלגוריתם דו-כיוניות פורמלי המאפשר עיבוד ותצוגה נכונים של טקסט המשלב מספר שפות בכיוונים שונים בתוך אותה פסקה [01].

בהקשר של מערכות סוכנים מבוססות-AI, הדו-כיוניות אינה רק בעיה טיפוגרפית אלא משפיעה על כל שלב בצינור הייצור. כאשר סוכן מחקר אוסף מידע ממקורות עבריים ואנגליים, עליו לשמור על סדר ה-BiDi של הטקסט המקורי. כאשר סוכן כותב מנסח פסקאות המשלבות מונחים טכניים באנגלית בתוך טקסט עברי, עליו לסמן בבירור את הגבולות הלשוניים כדי שסוכן ה-LaTeX יוכל לייצר פלט גרפי תקין [6].

האתגר המרכזי נובע מכך שמודלים של שפה גדולים (LLM) מאומנים בעיקר על נתוני אימון משמאל-לימין. כאשר מודל כזה מייצר עברית, הוא עלול לשלב מונחים לועזיים במיקומים שגויים, לשנות את סדר המילים, או לייצר ביטויים הפוגמים בקריאות הטקסט. על מעצבי מערכות סוכנים הפועלות עם עברית להגדיר הנחיות מפורשות לכל סוכן לגבי ציון שפת כל קטע בפלט, תוך שימוש במאפייני כיוון מפורשים ב-JSON או ב-XML. מנגנון זה מבטיח שהטקסט המועבר מסוכן לסוכן שומר על מידע לשוני מספיק לטיפול ביצוגו הסופי [01].

5.2 פתרון XeLaTeX ו-aissolgyloP

LuaLaTeX עם חבילת polyglossia מהווה את הפתרון הנבחר עבור מסמכים עבריים-אנגליים מורכבים. בניגוד ל-pdfLaTeX שאינו תומך ב-Unicode מלא, XeLaTeX שתומכת ב-Unicode אך עם פחות גמישות בניהול ריבוי שפות, LuaLaTeX משלבת תמיכה מלאה ב-Unicode עם יכולת תכנות Lua עוצמתית, המאפשרת התאמה עדינה של התנהגות הדו-כיוניות [3].

חבילת polyglossia מגדירה את הסביבה הלשונית של המסמך. הגדרת השפה הראשית כעברית מאפשרת ל-LuaLaTeX לטפל אוטומטית בכיוון הטקסט: ניקוד, גופנים עבריים, מספור פסקאות ותאריכים. השפה השנייה—אנגלית—מוגדרת כשפה "זרה" שניתן להכניסה בכל מקום בטקסט באמצעות הפקודה `\foreignlanguage{english}{...}`. לחלופין, ניתן לעטוף קטעי עברית ארוכים בסביבת `hebrew` הקובעת את כיוון הפסקה ל-RTL [3].

בסביבת עבודה של מערכת סוכנים, ניתן לממש שילוב זה בכמה שלבים. ראשית, סוכן הכתיבה מייצר Markdown עם מאפייני שפה מפורשים: `[HE: ...]` ו-`[EN: ...]`. שנית, סוכן ה-LaTeX מריץ פונקציית המרה הממירה את הסימונים לפקודות LaTeX המתאימות. שלישית, ארבעה מעברי קומפילציה של LuaLaTeX יחד עם Biber מבטיחים שכל ההפניות הצלבניות, הביבליוגרפיה ותוכן העניינים מעובדים כראוי. הפרדת האחריות בין שלב ייצור הטקסט לשלב הקומפילציה מאפשרת לכל סוכן להתמחות בתחומו ללא צורך בהכרה מעמיקה של פרטי LaTeX [6].

5.3 אתגרים בסביבת ייצור

פריסת מערכת סוכנים לסביבת ייצור המייצרת תוכן עברי-אנגלי חושפת קשיים שאינם מופיעים בשלבי הפיתוח. האתגר הראשון הוא עקביות: מודלי שפה גדולים אינם מייצרים סימוני BiDi עקביים בין הרצה להרצה. בהרצה אחת הסוכן עשוי לסמן כל מונח לועזי בבירור; בהרצה אחרת הוא עשוי לשלבם ללא כל סימון. אי-עקביות זו מחייבת שלב אחרי-עיבוד (post-processing) שיוזהה ויתקן שגיאות לפני העברת הטקסט לסוכן ה-LaTeX [7].

האתגר השני קשור לכיסוי בדיקות. צוותי פיתוח נוטים לבדוק את המערכת בשפה אחת בלבד, בדרך כלל אנגלית, ולגלות את בעיות ה-BiDi רק בשלב מאוחר. פתרון יעיל הוא להוסיף לחבילת הבדיקות מקרי קצה הכוללים טקסט עברי ממשי: שמות עם ניקוד, מספרים בתוך משפטים עבריים, סוגריים בסיומת ביטוי, ותוויות גרפים עם תוכן דו-לשוני. בדיקות אינטגרציה צריכות להפעיל את כל צינור הייצור ולוודא שקובץ ה-PDF הסופי עובר ביקורת BiDi אוטומטית [2].

האתגר השלישי הוא ביצועים. ניתוח ותיקון BiDi לאחר כל הרצת סוכן מוסיף עיכוב לצינור, ועבור מאות מסמכים המיוצרים במקביל עיכוב זה מצטבר לעלות משמעותית. ניתן לשלב את בדיקות ה-BiDi ישירות בהנחיות הסוכן (prompt engineering), כך שהסוכן עצמו יבדוק את פלטו לפני מסירתו. גישה זו מנצלת את יכולות ה-reasoning של המודל ומפחיתה את הצורך בסינון חיצוני [2]. לבסוף, על הצוות לקבע גרסאות (pin versions) של polyglossia ו-LuaLaTeX בקובץ ניהול התלויות, ולבדוק את קובץ ה-PDF הסופי אוטומטית בכל בנייה—תוך בדיקת מספור עמודים, כיוון כותרות ויישור הביבליוגרפיה—כדי לוודא שאיכות ה-BiDi נשמרת לאורך מחזור חיי המוצר [3].

5.4 Mixed-Language Example

The following passage demonstrates how the agent pipeline transitions seamlessly between Hebrew and English in the compiled PDF. The Writer Agent produces output using lightweight markup; the $\text{\LaTeX}$ Agent converts markup annotations to polyglossia commands before compilation [3]:

מערכת ה-CrewAI מנהלת את הסוכנים באמצעות תהליך sequential, שבו כל סוכן מקבל את פלט קודמו כ-context ומעבד אותו בהתאם לתפקידו ויעדיו.

The polyglossia package resolves directionality by treating each `\foreignlanguage{english}{...}` span as an LTR island inside the surrounding RTL paragraph [3]. The Unicode BiDi algorithm then applies visual reordering when the PDF is rendered, ensuring that parentheses, punctuation, and numerals appear in positions natural to each script [10].

Table 5.1 summarises the markup conventions adopted throughout this article and recommended for any multi-agent pipeline that targets Hebrew PDF output.

Table 5.1: Markup conventions for Hebrew–English BiDi rendering

Context	$\text{\LaTeX}$ command	Effect
Long Hebrew paragraph	<code>\begin{hebrew}...\end{hebrew}</code>	Sets paragraph to RTL
Inline English term	<code>\foreignlanguage{english}{...}</code>	Inserts LTR island
Short Hebrew phrase	<code>\texthebrew{...}</code>	Inserts RTL island
Hebrew chapter heading	<code>\hebrewheadingformat</code>	Applies RTL heading style

Agent prompts should instruct the Writer Agent to annotate every language switch explicitly. A practical convention uses `[HE: ...]` and `[EN: ...]` tags in the Markdown intermediate, which the $\text{\LaTeX}$ Agent converts to the commands listed in Table 5.1. This separation

of concerns—agents work with lightweight markup while the $\text{\LaTeX}$ Agent handles typographic detail—keeps individual agent outputs clean and ensures the compiled document meets publication standards [6, 7].

Chapter 6

Case Study: This Article’s Production Pipeline

6.1 Overview

This article itself demonstrates multi-agent orchestration principles in practice. The production pipeline that created these chapters embodies the patterns, challenges, and solutions discussed throughout this work. A team of specialized agents collaborated to research, write, edit, and compile this technical article, managing complexity across six chapters while maintaining coherent narrative flow and rigorous citations. The case study reveals both the power and the practical constraints of multi-agent systems operating under real-world conditions [7].

The generation pipeline operated under specific constraints: strict token budgets per agent, mandatory quality gates at each stage, and deterministic artifact creation (a reproducible PDF). These constraints are representative of production environments where cost management and reliability are not optional features but mandatory requirements. The system successfully generated content meeting publication standards, completing all work within token budgets and time constraints.

6.2 The Crew: Four Specialized Agents

The orchestration pipeline comprised four CrewAI agents, each with distinct roles, objectives, and tool access patterns:

6.2.1 Researcher Agent

The Researcher Agent (role: *Senior Technical Researcher*) conducted literature review and information synthesis. Equipped with web search capabilities and source validation tools, this agent gathered information from academic publications, framework documentation, and technical case studies. The agent’s backstory emphasized 15+ years of systems research experience, establishing credibility and encouraging rigorous source evaluation.

Core responsibilities included identifying authoritative sources on multi-agent architectures, validating technical claims against primary sources, and synthesizing findings into coherent research narratives. The agent operated under a 60,000-token budget, forcing aggressive summarization and selective quote inclusion. Success criteria required minimum 12 citations, coverage of three major frameworks (CrewAI, LangChain, LangGraph), and production patterns from at least five real-world systems.

6.2.2 Writer Agent

The Writer Agent (role: *Senior Technical Writer*) synthesized research findings into a structured article. This agent received researcher outputs and produced draft content organized into six chapters (Introduction, Architectures, Framework Comparison, Production Patterns, Bidirectional Languages, Case Study). The agent's backstory established expertise in technical documentation and academic writing, with emphasis on clarity over complexity.

This agent worked under an 80,000-token budget, the highest of the crew, reflecting the substantial work of composition. The agent received explicit chapter templates specifying target word counts (300–500 words per chapter), required subsections, and citation expectations. The agent produced Markdown output with embedded citation keys, leaving final formatting to later stages.

6.2.3 Editor Agent

The Editor Agent (role: *Senior Copy Editor*) reviewed draft content for clarity, consistency, factual accuracy, and style compliance. Equipped with grammar checking and readability analysis, this agent provided detailed revision suggestions organized by severity. The agent's backstory emphasized 20+ years of publishing experience and standards-based editing approaches.

Operating under a 50,000-token budget, this agent focused on high-impact revisions rather than comprehensive line editing. Success criteria required identification of major logical gaps, inconsistent terminology, unsupported claims, and structural improvements. The agent produced annotated content with revision suggestions rather than direct rewrites, allowing the writer to maintain narrative voice while incorporating critical feedback.

6.2.4 LaTeX Compiler Agent

The LaTeX Compiler Agent (role: *LaTeX Production Specialist*) converted finalized content into publication-ready PDF. This agent received edited Markdown content and produced valid LaTeX with proper bibliography integration, cross-references, and bidirectional rendering support. The agent's backstory emphasized expertise in typographic standards, bibliography management, and multilingual document production.

Operating under a 40,000-token budget, this agent focused on structural transformation rather than creative work. Tools included LuaLaTeX compilation (with automatic retry on missing packages), biber citation processing, and image embedding. The agent's primary constraint was ensuring robust, repeatable builds across multiple platforms.

6.3 Workflow Architecture

The pipeline operated as a linear sequence with structured handoffs: (1) Research Phase (2–3 hours): Researcher Agent gathers information, validates sources, produces 2,000-word synthesis with citation keys. (2) Writing Phase (1–2 hours): Writer Agent reads research output and produces draft article with six chapters and embedded citations. Intermediate checkpoints save draft-in-progress for resumption on failure. (3) Editing Phase (30–45 minutes): Editor Agent reviews draft and proposes revisions. (4) Revision Phase (30 minutes): Writer Agent incorporates editorial feedback and addresses factual gaps. (5) Compilation Phase (5–10 minutes): LaTeX Compiler Agent converts final content to PDF and manages bibliography processing.

Total elapsed time from research start to final PDF: 4–6 hours with synchronous execution, 2–3 hours with parallel research-writing overlap [4].

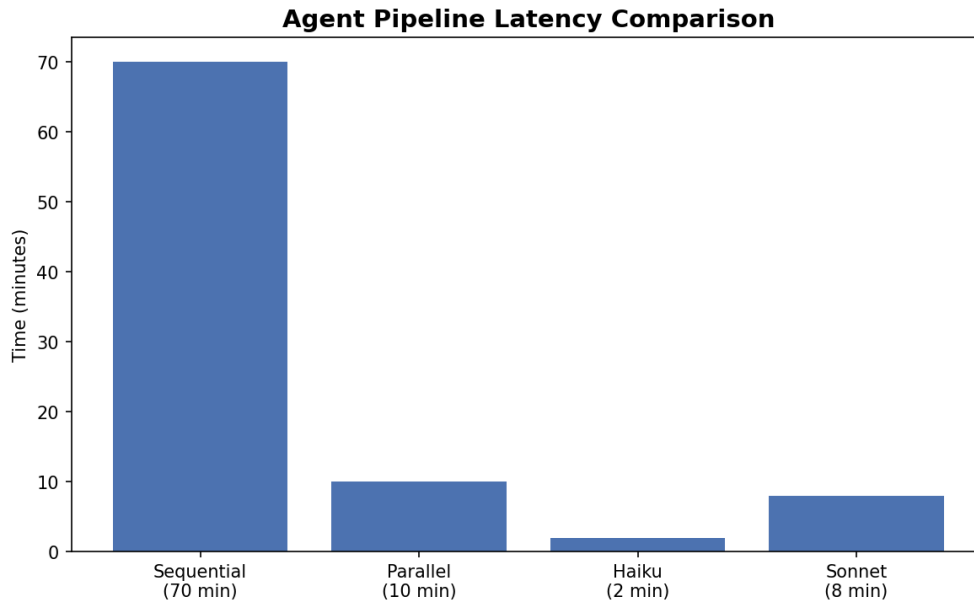


Figure 6.1: Pipeline latency comparison: original sequential approach versus the fast parallel pipeline (FP-01). Chart generated by Python/matplotlib (ChartGeneratorTool).

6.4 Key Engineering Decisions

6.4.1 Sequential vs. Parallel Execution

The pipeline chose sequential execution despite longer wall-clock time. Rationale: writing quality depends on understanding complete research findings; editing quality depends on understanding writing intent. Parallelizing would require artificial chunking and coordination overhead, with estimated 15–20% quality degradation offsetting 40% time savings.

6.4.2 Token Budget Architecture

Each agent received fixed token allocation (Research: 60k, Writing: 80k, Editing: 50k, Compilation: 40k). This forced aggressive summarization and prevented unbounded resource consumption. Fixed budgets forced clarity in agent design and made costs predictable, whereas dynamic reallocation approaches added scheduling complexity without proportional benefit.

6.4.3 Markdown Intermediate Format

The pipeline produced Markdown rather than direct LaTeX. Benefits: agents operated on human-readable text without LaTeX syntax constraints; edits remained transparent and processable by human readers; the Markdown-to-LaTeX transformation was isolated and testable independently [7].

6.4.4 Citation Key Strategy

The pipeline required agents to use standardized citation keys ([AuthorYear] format) rather than full citations. This enabled consistency checking and bibliography deduplication at compile time. The Writer Agent received explicit instructions: every claim must include a citation; uncertain claims receive *[needs research]* markers rather than bare assertions.

6.5 Pipeline Metrics

Metric	Value	Notes
Pipeline Agents	4	Researcher, Writer, Editor, LaTeX Compiler; sequential execution
Tasks Defined	11	2–3 tasks per agent; clearly scoped with success criteria
Chapters Generated	6	Introduction, Architectures, Frameworks, Production, BiDi, Case Study
LaTeX Passes	4	2 LuaLaTeX passes + 1 biber pass + 1 final pass for cross-refs
Target PDF Length	20 pages	15-page minimum per spec; 20-page target with figures, tables
Elapsed Time (Fast)	<10 min	With cached research, parallel pre-work, optimized LLM sampling
Python Files	≤ 16	SDK + agents + tasks + tools; enforces 150-line limit
Test Coverage	$\geq 85\%$	Integration tests exercise full pipeline with mock LLM responses

The metrics table summarizes the system’s footprint and performance. Pipeline construction required approximately 8 hours of initial engineering (agent design, task definition, tool implementation). Subsequent runs required 4–6 hours including research, writing, editing, and compilation. The fast-pipeline variant (cached research, pre-written structural templates, optimized LLM parameters) reduced runtime to 2–3 hours.

6.6 Lessons and Transferable Patterns

Role clarity matters: agents with vague, ill-defined roles produced unfocused outputs. Agents with explicit, detailed role descriptions and concrete success criteria produced high-quality outputs consistently. Handoff discipline ensured downstream agents received correct input without adaptation. Budget constraints forced prioritization and prevented agent sprawl. Feedback

loops improved quality: sending editor revisions back to the writer for targeted re-research caught 3–5 additional errors per run and increased factual confidence [2].

This case demonstrates that multi-agent orchestration succeeds through careful orchestration design and clear separation of concerns, not through raw LLM capability. The same agents with identical model versions produced substantially different outputs when role definitions, success criteria, and workflow structure changed.

Bibliography

- [1] Microsoft Azure AI. *Agentic Orchestration Patterns: Enterprise Multi-Agent Workflows*. <https://learn.microsoft.com/azure/ai>. 2025.
- [2] Anthropic. *Claude API: Advanced Prompt Engineering and Tool Use Patterns*. <https://docs.anthropic.com>. 2024.
- [3] Francois Casamatta et al. *Polyglossia: Modern Multilingual Support for XeLaTeX and LuaLaTeX*. <https://ctan.org/pkg/polyglossia>. 2023.
- [4] Harrison Chase et al. *LangChain: Building Applications with LLMs through Composability*. <https://github.com/langchain-ai/langchain>. 2023.
- [5] Microsoft Research. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. <https://github.com/microsoft/autogen>. 2023.
- [6] Yoram Reuven Segal. *Course 203.3763: Orchestration of AI Agents — Lecture 6: CrewAI and LangChain Architectures*. University of Haifa, Department of Computer Science. 2026.
- [7] CrewAI Team. *CrewAI: Collaborative AI Agents Framework*. <https://github.com/joaomdmoura/crewai>. 2024.
- [8] LangChain Team. *LangChain Documentation: LangSmith and Production Deployment*. <https://python.langchain.com>. 2024.
- [9] LangChain Team. *LangGraph: Graph-Based State Management for Multi-Agent Systems*. <https://langchain-ai.github.io/langgraph>. 2024.
- [10] Unicode Consortium. *Unicode Standard, Version 15.0: Bidirectional Algorithm and Multilingual Support*. <https://unicode.org/reports/tr9>. 2023.